Operadores

Escola Tecnológica FPFtech

Curso: Técnico em Automação Industrial

Linguagem de Programação Voltada a Microcontroladores

Semana 01

Prof. Denis Moraes Guimarães

24/09/2026

Objetivos e percurso da aula

Ao final, você deverá conseguir

Construir expressões aritméticas, relacionais e lógicas em C, prever seus resultados e explicar os principais cuidados.

1. Calcular grandezas a partir das variáveis.
2. Comparar valores e combinar condições.
3. Interpretar precedência e curto-circuito.
4. Simular uma decisão de automação com expressões.

Uma expressão produz um valor

Situação: uma linha de embalagem

Foram produzidas 17 peças. Cada caixa comporta 5 peças.
Quantas caixas completas e quantas peças restantes existem?



Operando: valor sobre o qual se opera.
Operador: símbolo que especifica a operação.
Em `17 / 5`, os operandos são 17 e 5; o operador é `/`.

Operadores aritméticos

Operador	Operação	Exemplo e resultado
+	Adição	8 + 3 resulta em 11
-	Subtração	8 - 3 resulta em 5
*	Multiplicação	8 * 3 resulta em 24
/	Divisão	8 / 3 resulta em 2
%	Resto inteiro	8 % 3 resulta em 2

O tipo dos operandos importa

Inteiros e valores de ponto flutuante podem produzir resultados diferentes na divisão. O operador de resto exige operandos inteiros.

Divisão inteira e resto na embalagem

```c
int pecas = 17;
int capacidade = 5;
int caixas_completas = pecas / capacidade;
int pecas_restantes = pecas % capacidade;
```

Expressão	Resultado
17 / 5	3 caixas completas
17 % 5	2 peças restantes

Conferência: $17 = 3 \times 5 + 2$. Para valores positivos, o resto fica entre zero e um a menos que o divisor.

Divisão real: converter antes de dividir

```c
int pecas = 17;
int capacidade = 5;
double a = pecas / capacidade;          /* 3.0 */
double b = (double)pecas / capacidade; /* 3.4 */
double c = 17.0 / 5.0;                  /* 3.4 */
```

- Em `a`, a divisão inteira ocorre primeiro.
- Armazenar o resultado em `double` não recupera a fração.
- Em `b`, o *cast* altera o tipo de um operando antes da operação; o outro é convertido para `double`.

Sinais, divisão por zero e limites

- Em C11, a divisão inteira descarta a parte fracionária em direção a zero: `-7 / 3` resulta em -2.
- O resto acompanha o sinal do dividendo: `-7 % 3` resulta em -1.
- A divisão inteira e o resto com divisor zero têm comportamento indefinido: valide o divisor.
- Um resultado fora da faixa de um inteiro com sinal também pode causar comportamento indefinido.

Prática desta aula

Usaremos operandos pequenos e divisores conhecidos. Para ponto flutuante, também não devemos depender de uma divisão por zero.

Expressões para grandezas de automação

```
double tempo_s = 12.0;
double distancia_m = 3.0;
double velocidade_m_s = distancia_m / tempo_s;
double tempo_ms = tempo_s * 1000.0;
```

- Velocidade: $3{,}0/12{,}0 = 0{,}25$ m/s.
- Conversão de tempo: $12{,}0 \times 1000 = 12000$ ms.
- Verifique se as unidades fazem sentido antes de confiar no número.

Hipótese do exemplo

Tempo positivo e medidas simuladas; ainda não há leitura de hardware.

Atribuição e atualização de uma variável

```c
int contador = 10;
contador = contador + 1;  /* passa para 11 */
contador += 4;            /* passa para 15 */
contador -= 2;            /* passa para 13 */
contador++;               /* passa para 14 */
```

- A expressão à direita é calculada antes de armazenar o novo valor.
- += e -= combinam operação com atribuição.
- `contador++` incrementa uma unidade; `contador--` decrementa.

Nesta etapa, use incrementos como instruções isoladas para facilitar o rastreamento.

Precedência: por onde começar o cálculo?

Ordem de agrupamento	Operadores desta aula
Primeiro	Parênteses para agrupar explicitamente
Depois	Unários: `!`, sinal `-`, sinal `+`
	`*`, `/`, `%`
	`+`, `-`
	`<`, `<=`, `>`, `>=`
	`==`, `!=`
	`&&`, depois `||`
Por último	Atribuições: `=`, `+=`, `-=`

Esta é uma seleção, não a tabela completa de C. Precedência define agrupamento; não estabelece uma ordem geral de avaliação dos operandos.

Rastreamento de expressões aritméticas

Expressão	Cálculo e resultado
`2 + 3 * 4`	$2 + 12 = 14$
`(2 + 3) * 4`	$5 \times 4 = 20$
`20 / 5 * 2`	$4 \times 2 = 8$
`(5 + 8) / 2`	$13/2 = 6$ (inteiro)
`(5 + 8) / 2.0`	$13/2{,}0 = 6{,}5$

Método

Identifique tipos, aplique os agrupamentos e só então determine o valor armazenado. Para operadores aritméticos binários no mesmo nível, o agrupamento é da esquerda para a direita.

Operadores relacionais: comparar valores

Operador	Significado	Exemplo verdadeiro
<	Menor que	20 < 30
<=	Menor ou igual	30 <= 30
>	Maior que	40 > 30
>=	Maior ou igual	30 >= 30
==	Igual a	5 == 5
!=	Diferente de	5 != 4

Em C, comparações produzem `int`: 1 para verdadeiro e 0 para falso. Podemos armazenar o resultado em `bool`.

Comparação em uma medição simulada

```
double temperatura_c = 65.0;
const double limite_c = 60.0;
bool acima_limite = temperatura_c > limite_c;
bool no_limite = temperatura_c == limite_c;
```

- `acima_limite` recebe verdadeiro.
- `no_limite` recebe falso.
- Comparar não altera o valor da temperatura.

Escolha do requisito

"Acima de 60" usa >; "a partir de 60" usa >=. A fronteira faz parte da especificação.

Atribuição não é teste de igualdade

```
int nivel = 8;
bool igual_a_dez = (nivel == 10); /* falso */
nivel = 10;                       /* altera nivel */
```

Operação	Propósito	Efeito no nível
`nivel == 10`	Comparar	Mantém 8
`nivel = 10`	Atribuir	Passa para 10

Leitura em voz alta

Leia = como “recebe” e == como “é igual a?”.

Operadores lógicos: combinar condições

Operador	Nome	Resultado verdadeiro quando
`&&`	E lógico	Ambas são verdadeiras
`||`	OU lógico	Ao menos uma é verdadeira
`!`	NÃO lógico	A condição é falsa

- Em um contexto lógico, zero é falso; qualquer valor diferente de zero é verdadeiro.
- `&&`, `||` e `!` produzem 0 ou 1, com tipo `int`.
- Prefira condições nomeadas com `bool` para expressar estados.

Tabela-verdade

A	B	A && B	A || B	!A
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Interpretação

Uma partida simulada pode exigir “comando ativo **E** proteção fechada”. Um aviso pode ocorrer por “temperatura alta **OU** nível baixo”.

São modelos didáticos de lógica; sistemas reais de proteção exigem projeto próprio.

Representar um intervalo corretamente

```
double temperatura_c = 25.0;
bool na_faixa = (temperatura_c >= 20.0)
              && (temperatura_c <= 30.0);
bool fora_faixa = !na_faixa;
```

- O intervalo inclui 20 e 30: os dois limites são inclusivos.
- Para 25, `na_faixa` é verdadeiro.
- Não escreva `20 <= temperatura_c <= 30`.

Por que a comparação encadeada falha?

C avalia `20 <= temperatura_c` primeiro, obtendo 0 ou 1. Depois compara esse resultado com 30.

Curto-circuito: nem sempre avaliar tudo

```
int quantidade = 0;
int total = 100;
bool media_alta = (quantidade != 0)
                && (total / quantidade > 20);
```

- `&&` avalia primeiro a esquerda. Se ela for falsa, não avalia a direita.
- No exemplo, a divisão não ocorre; `media_alta` é falso.
- `||` também começa à esquerda: se ela for verdadeira, não avalia a direita.

A ordem protege a operação

Verificar o divisor deve vir antes da divisão. Inverter as duas expressões elimina essa proteção.

E lógico não é E bit a bit

```c
int a = 2; /* binario: 10 */
int b = 1; /* binario: 01 */
int logico = a && b; /* 1: ambos nao sao zero */
int bits = a & b;   /* 0: nenhum bit 1 em comum */
```

- && combina a verdade das condições e tem curto-circuito.
- & combina bits de operandos inteiros e não tem curto-circuito.
- Não substitua && por & para "simplificar".

Operações de bits serão úteis em registradores; aqui, usamos operadores lógicos para combinar condições.

Combinar requisitos com nomes claros

```
bool comando = true;
bool protecao_fechada = true;
bool falha = false;
bool habilitado = comando && protecao_fechada && !falha;
```

Parte da expressão	Valor	Interpretação
`comando`	1	Há solicitação
`protecao_fechada`	1	Proteção sinalizada
`!falha`	1	Falha não sinalizada
`habilitado`	1	Todos os requisitos atendidos

O resultado é apenas um valor calculado; nenhum motor é acionado nesta simulação.

Erros comuns em expressões

Problema	Prática recomendada
Esperar 2,5 de `5 / 2`	Use `5.0 / 2.0` para divisão real.
Confundir = com ==	Diferencie atualização e comparação.
Usar `&` no lugar de `&&`	Escolha o operador pelo significado.
Escrever `a < x < b`	Combine duas comparações com `&&`.
Comparar resultados reais por igualdade exata	Defina uma tolerância conforme a aplicação.
Modificar uma variável várias vezes na mesma expressão	Separe as atualizações em instruções.

Atividade guiada: condição de operação simulada

1. Use `nivel_l = 180`, `temperatura_c = 28.0`, `comando = true` e `falha = false`.
2. Calcule se o nível é maior ou igual a 100 litros.
3. Calcule se a temperatura está entre 20 e 30 graus Celsius, incluindo os limites.
4. Combine: comando ativo, nível adequado, temperatura na faixa e ausência de falha.
5. Exiba os três resultados lógicos com `%d`.

Experimento

Altere apenas o nível para 99 e preveja os resultados antes de executar novamente.

Arquivo resolvido: `exemplos/03_operacao.c`.

Conferência por cenários

Considere comando verdadeiro e falha falsa:

Nível (L)	Temp. (°C)	Nível adequado	Na faixa	Habilitado
180	28,0	1	1	1
99	28,0	0	1	0
100	20,0	1	1	1
180	30,1	1	0	0

- Os limites exatos verificam o uso de >= e <=.
- Se `falha` passar a verdadeiro, o resultado final será falso, mesmo com nível e temperatura adequados.

Exercícios independentes

1. Preveja: `19 / 4`, `19 % 4`, `19 / 4.0`, `2 + 3 * 4` e `(2 + 3) * 4`.
2. Uma embalagem usa 6 peças. Para 29 peças, calcule caixas completas e peças restantes em C; exiba os resultados.
3. Declare `pressao_bar = 2.5`, `comando = true` e `falha = false`. Calcule `permitido` quando a pressão estiver entre 2 e 4 bar, inclusive, houver comando e não houver falha. Teste também 1,9 bar e 4,0 bar.
4. Com `int n = 0;`, explique por que `(n != 0) && (10 / n > 2)` é seguro e por que trocar `&&` por `&` elimina essa proteção.

Entrega

Respostas dos itens 1 e 4; programas e saídas dos itens 2 e 3. Registre a previsão antes de executar.

Síntese e ligação com controle de fluxo

- Operadores aritméticos calculam; relacionais comparam; lógicos combinam condições.
- O tipo dos operandos determina a divisão inteira ou real.
- Parênteses tornam a intenção visível.
- = atribui; == testa igualdade.
- Curto-circuito pode impedir uma operação inválida.

Bilhete de saída

Escreva uma expressão para "nível abaixo de 50 litros OU temperatura acima de 40 graus Celsius" e explique seu resultado no limite exato.

Na próxima aula, usaremos essas condições para escolher e repetir instruções.